

CLAUDE 101

Workshop Companion Handout

Keep this. Every how-to step, example prompt, and both hands-on exercises live here — so you don't need to take notes during the session.

1

Pick the right tool

Harshit Singhai

Six building blocks. When you want the thing on the left, reach for the thing on the right.

If you want...	Use...
A one-off question or a quick draft	Just chat
A specific job done your way, every time	A Skill
A whole toolkit for a role (skills + connectors + jobs)	A Plugin
Related work, files & context kept together over time	A Project
A reusable view you'll reopen (dashboard / tracker)	An Artifact
Something that should run again and again on its own	A Schedule
Claude to act on a website behind a login	Claude in Chrome

2

Files vs. connectors — your two sources of context

Harshit Singhai

In Cowork, Claude works from a folder on your computer and/or live connectors. Most real tasks use both.

Files (a folder)

GOOD FOR

- Creating & editing real deliverables (decks, sheets, docs)
- Local, private content; no setup needed
- Bulk or messy files you want cleaned/reorganised

WATCH OUT

- Must be on your machine; you keep it current
- Big folders can eat tokens — point to specifics

Connectors (live tools)

GOOD FOR

- Always-current data — pulls the latest, no stale copy
- Reach across tools (Drive + Slack + Calendar)
- "Point, don't paste" — grab just what's needed

WATCH OUT

- One-time sign-in; some are admin-gated
- Only reads what you authorise; needs network

Rule of thumb: use connectors to *pull live context in*, and a folder to *create and edit the things you ship*.

3

How to actually do it

Harshit Singhai

Connect Google Drive (or any connector)

1. Open the connectors / registry area in the Claude app.
2. Find Google Drive (or Slack, Gmail, Calendar) and click Connect.
3. Sign in with your Meesho account and approve access. One time only.
4. Then reference files by name: "Use the Q2 planning doc and pull out the risks."

Tip: if a tool you need isn't listed, ask Claude — it can suggest a connector for it.

Create your own skill

1. Describe the job to Claude: what it's for, the steps, and the exact output format.
2. Ask it to "create a skill" and give it a short name.
3. Test on a real (non-confidential) example; refine: "shorter," "add a section," "change the tone."
4. Save it. From then on it loads automatically when the job comes up.

No code needed. A skill is just clear instructions: when to use it, the steps, and the format.

Find & use shared skills and plugins

Already shared: skills/plugins your team publishes show up in your Claude — e.g. Meesho wiki memory, PM interview feedback, MoM tracker.

To use one: just start the relevant task — the skill fires on its own when it's relevant.

To add more: browse the plugin marketplace and your internal kits; install the ones that fit your role.

Create a plugin (bundle skills for the team)

1. Decide which skills belong together for a role or workflow.
2. Ask Claude to bundle them into a plugin, plus any connectors and scheduled jobs they need.
3. Package it and share it with your pod / publish to your internal kit.

When is a plugin worth it? When several people need the same workflow. One skill = your recipe; a plugin = the shareable recipe book.

Build an artifact

1. Ask for a reusable view: "Build me a tracker for my open tasks that I can reopen and refresh."
2. It appears in the sidebar and stays there between sessions.
3. Reopen any time and say "refresh with the latest data."

Schedule a job

1. Describe the cadence: "Every weekday at 8am, summarise my unread email," or "remind me in an hour."
2. Confirm it; it runs in the background and delivers the result.
3. Connector + schedule + artifact = a dashboard that updates itself.

Projects, memory & updating files

Projects: keep related chats, files, and standing instructions together — share with your pod so everyone works from one playbook.

Memory: tell Claude what's worth remembering (your team, formats, ongoing work); review and edit it any time.

Updating files: in Cowork, Claude edits real files in place — keep an original, then iterate ("tighter," "add a column").

4

Copy-paste example prompts

Harshit Singhai

The pattern that makes these work: Context + Example + Format. Swap in your own details.

Summarise a long doc

Read [doc name] and give me a one-page brief: 3 key points, 3 risks, 3 next steps. Plain language, no jargon.

Draft a leadership update

Draft a 5-line update for leadership on the [project] launch. Tone: confident, factual. Include what shipped, the impact metric, one risk, and the next step. Match the format of last month's update in [doc].

Turn notes into slides

Turn the attached meeting notes into a 6-slide outline: title, context, 3 decisions, owners & dates. One idea per slide.

Clean a spreadsheet

In [file].xlsx: remove blank rows, format column C as ₹ currency, add a total row and a month-on-month % change column. Keep a copy of the original.

4

Copy-paste example prompts (continued)

Harshit Singhai

Catch up on Slack

Summarise the last 7 days in [channel]: key decisions, open questions, and anything that needs my input. Bullet points.

Prep for a meeting

I have a meeting on [topic] at 3pm. Using [doc] and my calendar invite, give me talking points, likely questions, and 3 things to decide.

Compare two options

Compare [option A] and [option B] for [goal] in a table: pros, cons, cost, effort, and your recommendation with a one-line reason.

Rewrite for tone

Rewrite this message to be warmer and more concise, under 80 words, friendly but professional: [paste text].

4

Copy-paste example prompts (continued)

Harshit Singhai

Plan before doing

Before writing anything, give me a short plan for how you'll approach [task]. I'll confirm, then you do it.

Extract action items

From these notes, list only the action items as: owner – task – due date. Flag any without an owner.

Claude Code — terminal/IDE agent for engineering work. Writes, edits, runs, and debugs code across multiple files in your actual repo. Test-fix loops, git operations, real codebase changes.

Claude Cowork — desktop agent for non-developers. Automates file and task work without code: organizing folders, drafting decks/docs, running repeatable multi-step jobs across tools.

Design — Claude's UI/frontend generation capability, used inside Code, Cowork, and artifacts to produce mockups and polished visual output.

MCPs available online (Claude Marketplace) — Model Context Protocol servers extend Claude's reach (Asana, Jira, Figma, etc.). Marketplace = browsable directory of ready-made connectors/plugins beyond defaults.

→ *Online available design systems*

A browser extension that lets Claude see and act on the page you're on. Best when the data or action lives behind a login no connector reaches.

Real use cases

- Pull figures off an internal dashboard that has no connector.
- Summarise a long article or PDF open in a tab.
- Fill a long or repetitive web form.
- Compare options across several open tabs.
- Gather details from an admin tool and drop them into a doc.

Keep in mind

It works in your browser with your logged-in sessions; **you can watch it and step in.**

Prefer a connector when one exists — it's cleaner and more reliable than browsing.

- Review actions carefully on sensitive or irreversible pages (payments, deletes, sends).

7

Exercise 1 — build a skill (Slack status-post writer)

Vijan Soni

10 minutes. Everyone builds the same skill, then tests and tweaks it. Use the sample below — no confidential data.

Step 1 — Create the skill (paste this into Claude)

Create a skill called "slack-status".
When I paste rough project notes, write a Slack status update that has:

- a one-line TL;DR
- Done / In progress / Blockers
- Next

Tone: crisp, friendly, scannable.
Keep it under 120 words. Light emoji.
Then ask me to test it.

Step 2 — Test it with these sample notes

seller onboarding revamp

- new KYC flow live for 20% of sellers, error rate down
- bulk catalog upload still in QA, slipping ~3 days
- waiting on legal sign-off for the new T&C banner
- next: ramp KYC to 50%, finalise upload by Thu

Step 3 — Tweak it

- "Make it shorter — max 60 words."
- "Add a TL;DR line at the very top in bold."
- "Less formal, a bit more energy."
- "Drop the emoji — plain text only."

Step 4 — Share out & keep it

Save this skill — you'll reuse it in Exercise 2. If yours isn't quite right, the fix is almost always: be more specific about the output (length, sections, tone).

8

Exercise 2 — build a plugin (package it for the team)

Vijan Soni

8 minutes. Turn the skill you just built into a small, shareable plugin. If creating a plugin isn't available on your plan, follow along with the group demo — the concept is the goal.

Step 1 — Bundle it (paste this into Claude)

Bundle my "slack-status" skill into a plugin called "pod-status-kit". Include:

- the slack-status skill
- a scheduled Friday 4pm reminder to draft my weekly status
- the Slack connector (optional)

Then package it so I can share it with my team, and tell me how.

Step 2 — Discuss

- When is a plugin worth it instead of just a skill? (When several people need the same workflow.)
- What else would your pod want in this kit?

Stretch goal

Add a second skill (e.g. a meeting-notes → MoM skill) so it becomes a real "pod status kit," then share it.

Context on tokens limits and context windows

Optimise context & tokens

Fresh chat per topic — old context isn't free.

Point, don't paste — reference Drive/folders; attach only the pages that matter.

Ask for just what you need — "just the summary," not the whole essay.

Let it summarise-then-continue on long work instead of carrying everything.

Right-size the model — Haiku/Sonnet for routine work; Opus for the genuinely hard.

Share across the team

Projects hold shared instructions + files, so a pod works from one playbook.

Skills & plugins spread "how we do it" — build once, everyone benefits, output stays consistent.

Memory keeps your personal preferences; keep it tidy and reviewed.

Updating files — Claude edits real files in place; keep an original and iterate.

10 Resources & next steps

Product docs: docs.claude.com

Help & how-tos: support.claude.com

Prompting guide: docs.claude.com/.../prompt-engineering/overview

Internal: your Meesho plugins — wiki memory, PM interview feedback, MoM tracker.

This week: use the skill you built, share it with your pod, and pick one weekly task to schedule.

Which model to use

Model	Reach for it when...
Opus 4.8	The task is genuinely hard — deep reasoning, tricky analysis. Slower & pricier.
Sonnet 4.6	Your everyday default. Fast, strong, balanced. Start here.
Haiku 4.5	Quick, simple, high-volume jobs where speed & cost matter most.

Best practices

- instructions for Claude
- State goal, constraints, exact output format — vague input gets vague output - prompt engineering - example prompting, one-shot prompting etc
- One example beats a paragraph of description
- Structure long instructions with headers Claude can scan, not a wall of text
- Iterate like a teammate: "shorter," "add X," "change tone" — refine, don't rewrite

Open for Questions



We Value Your Feedback!

